

Deep Learning

Limitation of machine learning

1. Data Acquisition

The whole concept of machine learning is about identifying useful data. The outcome will be incorrect if a credible data source is not provided. The quality of the data is also significant. If the user or institution needs more quality data, wait for it. It will cause delays in providing the output. So, machine learning significantly depends on the data and its quality.

2. Time and Resources

The data that machines process remains huge in quantity and differs greatly. Machines require time so that their algorithm can adjust to the environment and learn it. Trials runs are held to check the accuracy and reliability of the machine. It requires massive and expensive resources and high-quality expertise to set up that quality of infrastructure. Trials runs are costly as they would cost in terms of time and expenses.

3. Structured Data

ML Algorithms works only on structured data

4. Results Interpretations

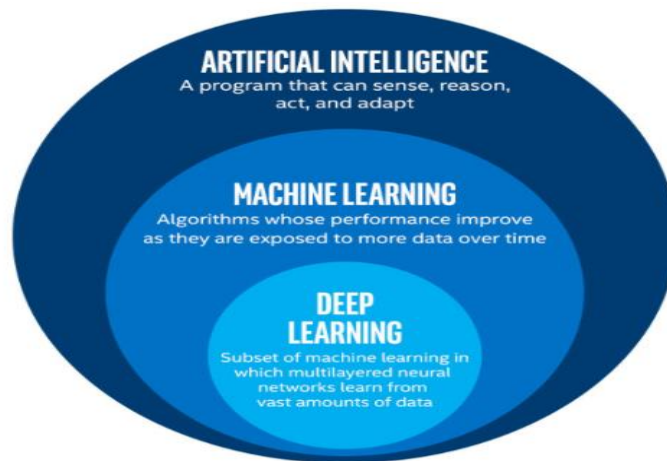
One of the biggest advantages of Machine learning is that interpreted data that we get cannot be hundred percent accurate. It will have some degree of inaccuracy.

5. High Error Chances

The error committed during the initial stages is huge, and if not corrected at that time, it creates havoc. Biasness and wrongness have to be dealt with separately; they are not interconnected. Machine learning depends on two factors, **i.e., data and algorithm**. All the errors are dependent on the two variables. Any incorrectness in any variables would have huge repercussions on the output.

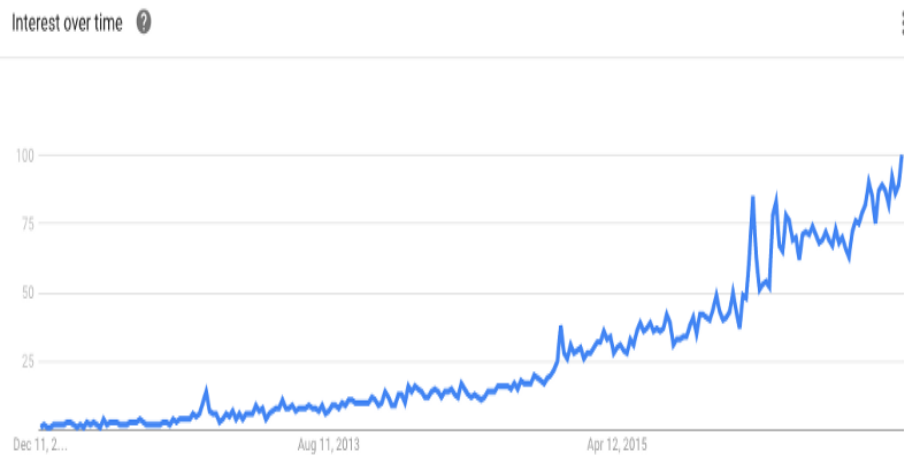
What is Deep Learning

Deep Learning is a subset of Machine Learning, which on the other hand is a subset of Artificial Intelligence. Artificial Intelligence is a general term that refers to techniques that enable computers to mimic human behaviour. Machine Learning represents a set of algorithms trained on data that make all this possible.

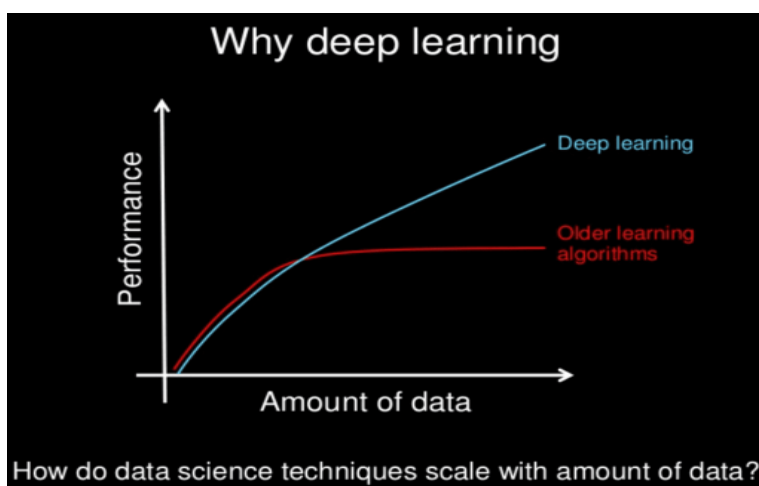


Why is it so popular?

- Exponential growth in DATA



- Performance



- Technology Upgradation

Cheap Hardware, GPU(Both local and cloud)

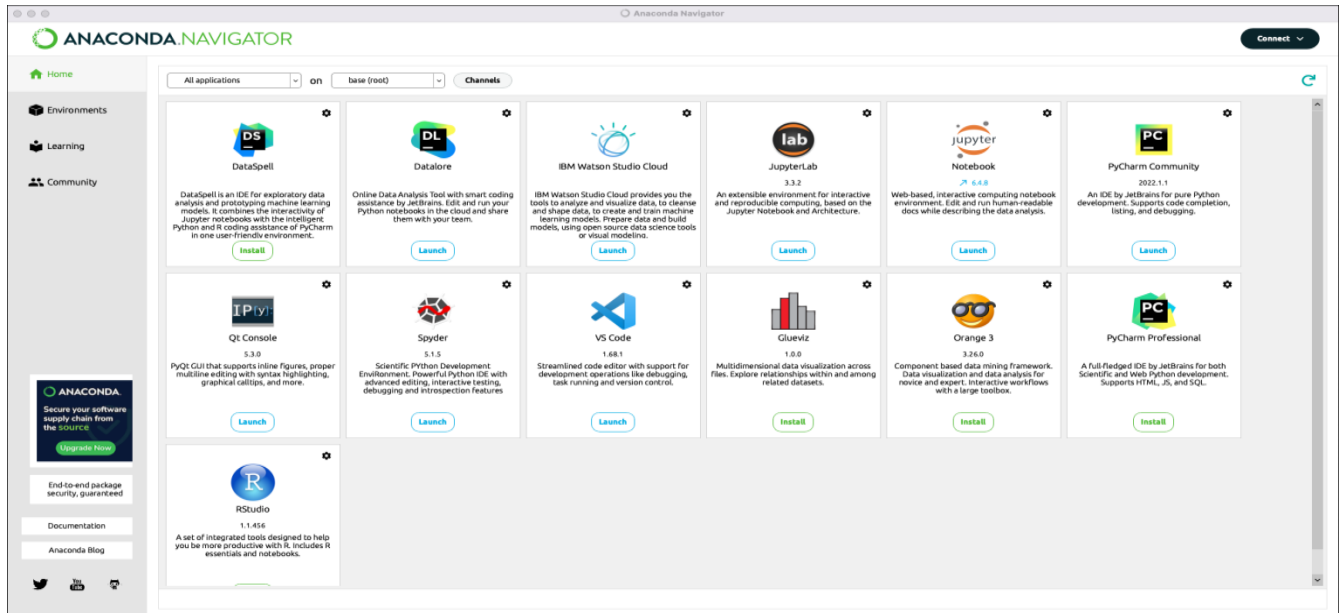
- No Feature Extraction

ML: DATA → FE and FS → TRAIN → OUTPUT

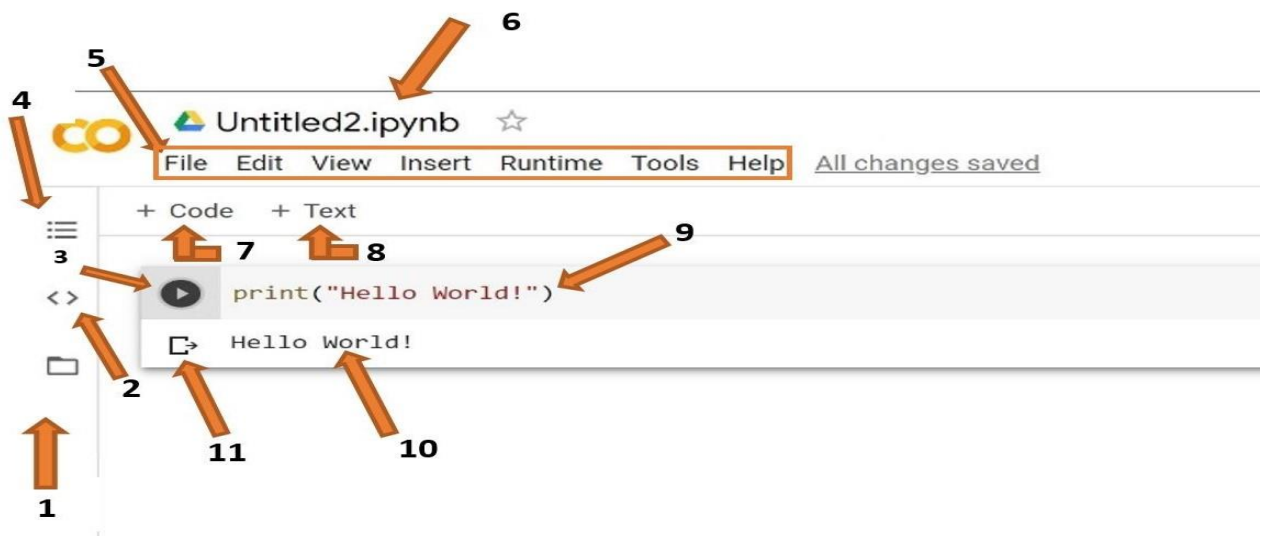
DL: DATA → TRAIN → OUTPUT

Environment Setup

Local(Anaconda)



Global (Google Colab)



Create Virtual environment

- ☐ Open anaconda prompt
- ☐ Check if conda is installed in your path

conda -V**▣ Set up the virtual environment**

```
conda create -n envname python=x.x anaconda
```

▣ Check environments

```
conda info --envs
```

▣ Activating the virtual environment

```
conda activate envname
```

▣ Deactivating the virtual environment

```
conda deactivate
```

Lets install Necessary Libraries**▣ Tensorflow**

```
Pip install tensorflow==version
```

▣ Tensorboard

```
Pip install tensorboard==version
```

▣ Keras

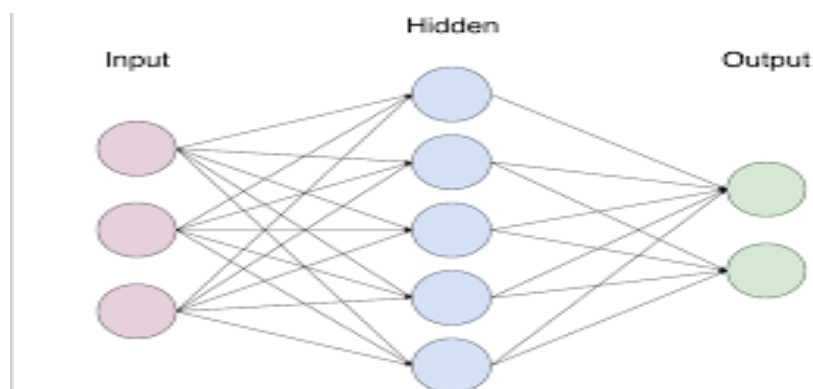
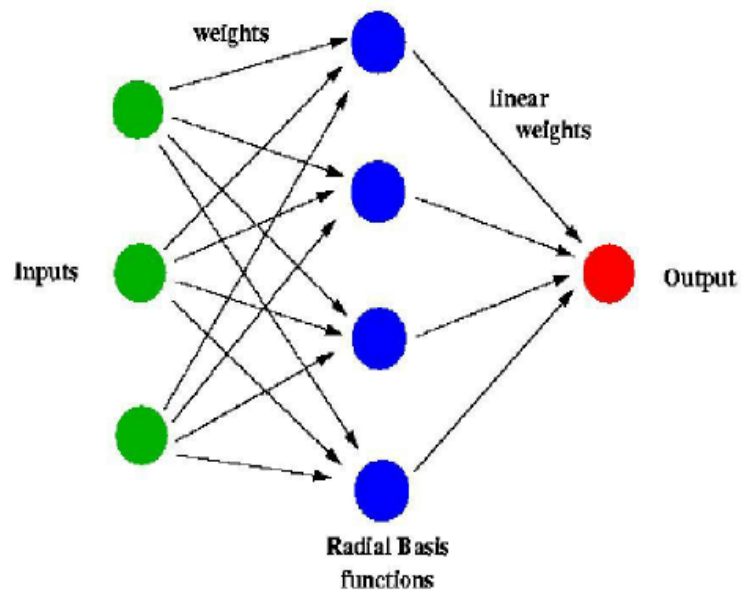
```
Pip install keras==version
```

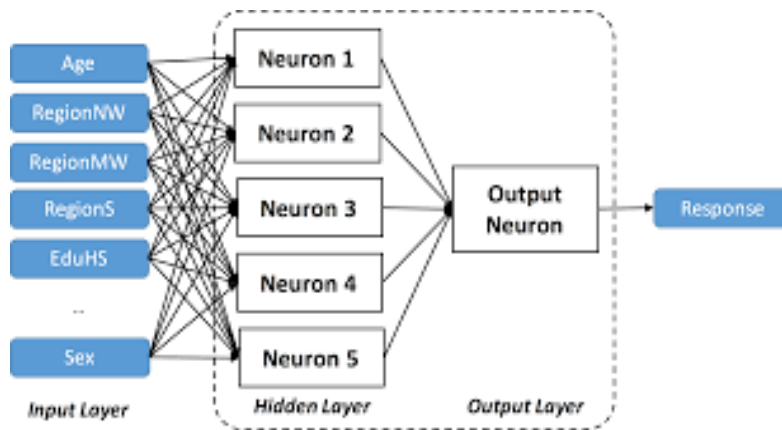
Introduction to Neural Network**What is Artificial Neural Network?**

- Artificial Neural Network ANN is an efficient computing system whose central theme is borrowed from the analogy of biological neural networks.
- ANN acquires a large collection of units that are interconnected in some pattern to allow communication between the units. These units, also referred to as nodes or neurons, are simple processors which operate in parallel.
- Every neuron is connected with other neuron through a connection link. Each connection link is associated with a weight that has information about the input signal.
- This is the most useful information for neurons to solve a particular problem because the weight usually excites or inhibits the signal that is being communicated.
- Each neuron has an internal state, which is called an activation signal.

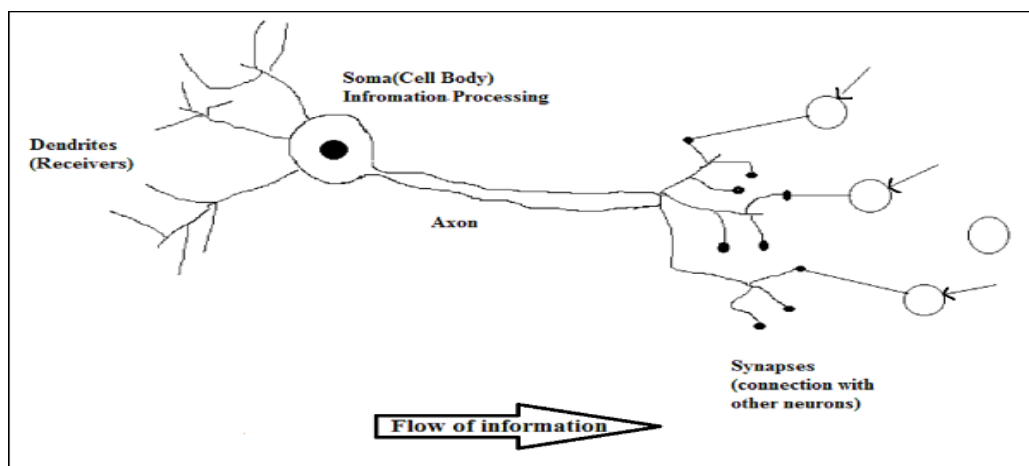
- Output signals, which are produced after combining the input signals and activation rule, may be sent to other units.

Example of Neural Networks





Biological Neuron



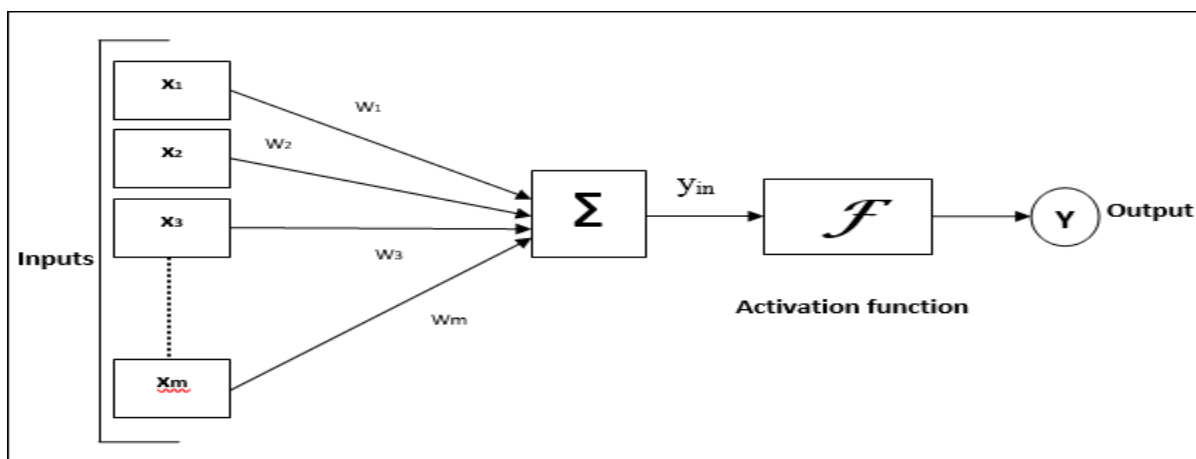
- A nerve cell neuron is a special biological cell that processes information. According to an estimation, there are huge number of neurons, approximately 10^{11} with numerous interconnections, approximately 10^{15} .
 - A typical neuron consists of the following four parts-
- **Dendrites** – They are tree-like branches, responsible for receiving the information from other neurons it is connected to. In other sense, we can say that they are like the ears of neuron.
- **Soma** – It is the cell body of the neuron and is responsible for processing of information, they have received from dendrites.
 - **Axon** – It is just like a cable through which neurons send the information.
 - **Synapses** – It is the connection between the axon and other neuron dendrites.

ANN versus BNN

Biological Neural Network BNN	Artificial Neural Network ANN
Soma	Node
Dendrites	Input
Synapse	Weights or Interconnections
Axon	Output

Model of Artificial Neural Network

- The diagram represents the general model of ANN followed by its processing.
- For the general model of artificial neural network, the net input can be calculated as follows –
 - $y_{in} = x_1 \cdot w_1 + x_2 \cdot w_2 + x_3 \cdot w_3 \dots x_m \cdot w_m$
 - i.e., Net input $y_{in} = \sum m_i x_i \cdot w_i$
- The output can be calculated by applying the activation function over the net input.
 - $Y = F(y_{in})$
 - Output = function net input calculated

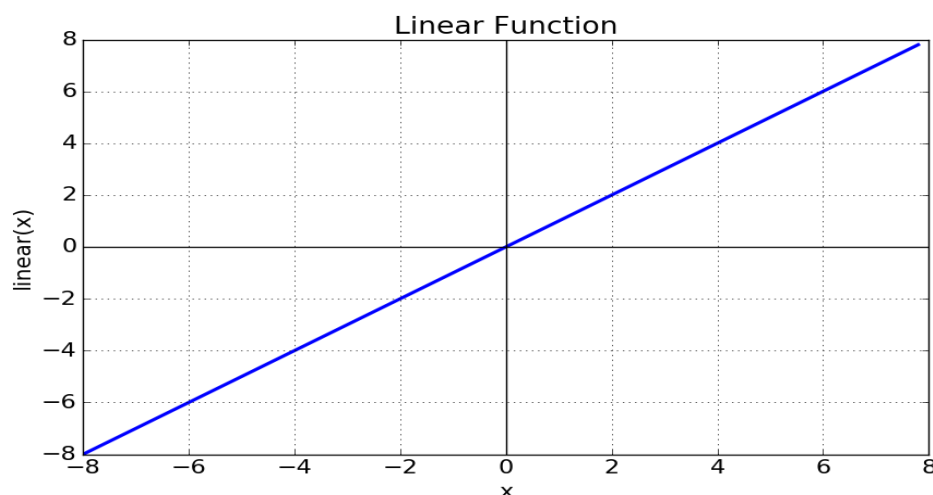
**What is Activation Function?**

- It's just a thing function that you use to get the output of node. It is also known as Transfer Function.

- Why we use Activation functions with Neural Networks?
 - It is used to determine the output of neural network like yes or no. It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).
- The Activation Functions can be basically divided into 2 types-
 - Linear Activation Function
 - Non-linear Activation Functions

Linear Activation Function

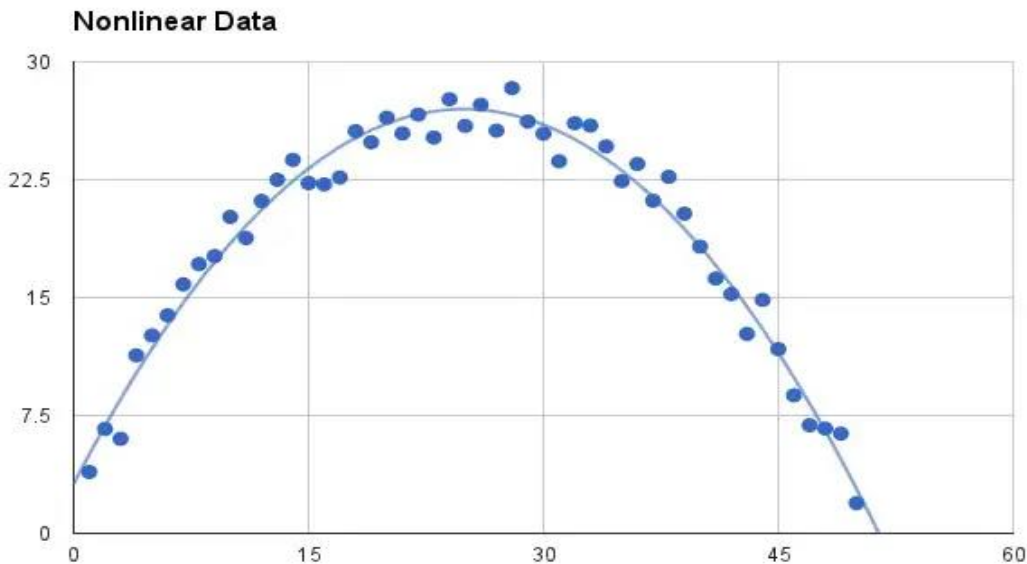
- Linear or Identity Activation Function
- As you can see the function is a line or linear. Therefore, the output of the functions will not be confined between any range.
- Equation : $f(x) = x$
- Range : (-infinity to infinity)
- It doesn't help with the complexity or various parameters of usual data that is fed to the neural networks.



Non-linear Activation Function

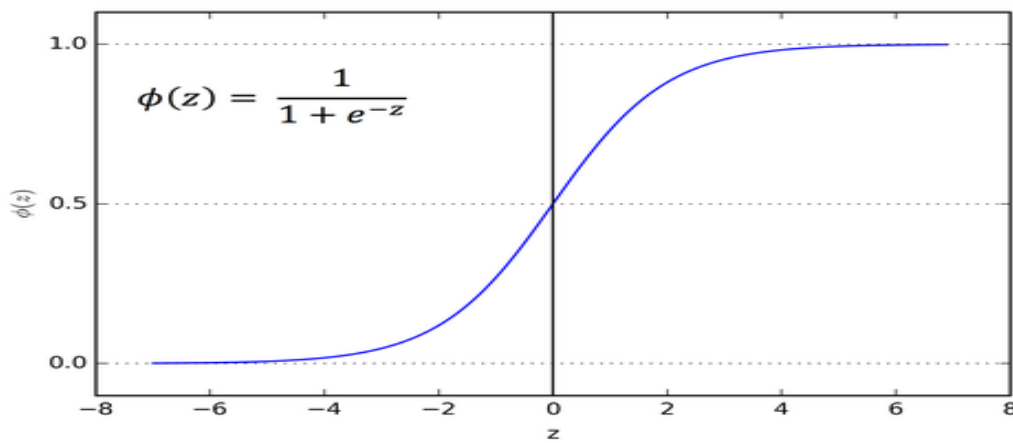
- The Nonlinear Activation Functions are the most used activation functions. Nonlinearity helps to makes the graph look something like this
- It makes it easy for the model to generalize or adapt with variety of data and to differentiate between the output.
- The main terminologies needed to understand for nonlinear functions are:
- Derivative or Differential: Change in y-axis w.r.t. change in x-axis. It is also known as slope.

- Monotonic function: A function which is either entirely non-increasing or non-decreasing.
- The Nonlinear Activation Functions are mainly divided on the basis of their range or curves



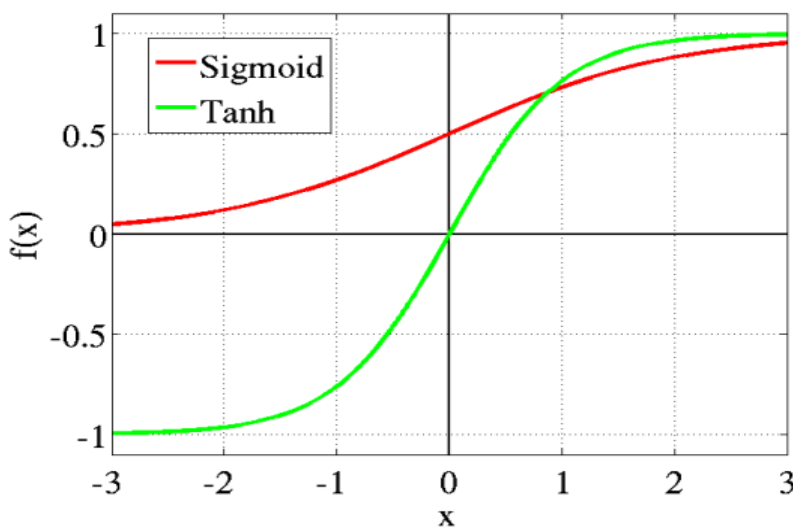
Sigmoid or Logistic Activation Function

- The Sigmoid Function curve looks like a S-shape.
- The main reason why we use sigmoid function is because it exists between **(0 to 1)**. Therefore, it is especially used for models where we have to **predict the probability** as an output. Since probability of anything exists only between the range of **0 and 1**, sigmoid is the right choice.
- The function is **differentiable**. That means, we can find the slope of the sigmoid curve at any two points.
- The function is **monotonic** but function's derivative is not.
- The logistic sigmoid function can cause a neural network to get stuck at the training time.
- The **softmax function** is a more generalized logistic activation function which is used for multiclass classification.



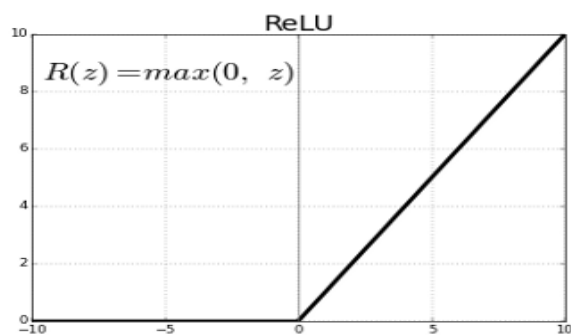
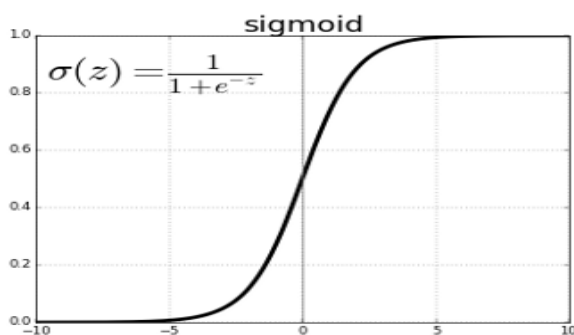
Tanh or hyperbolic tangent Activation Function

- tanh is also like logistic sigmoid but better. The range of the tanh function is from (-1 to 1). tanh is also sigmoidal (s - shaped).
- The advantage is that the negative inputs will be mapped strongly negative, and the zero inputs will be mapped near zero in the tanh graph.
- The function is differentiable.
- The function is monotonic while its derivative is not monotonic.
- The tanh function is mainly used classification between two classes.
- Both tanh and logistic sigmoid activation functions are used in feed-forward nets.



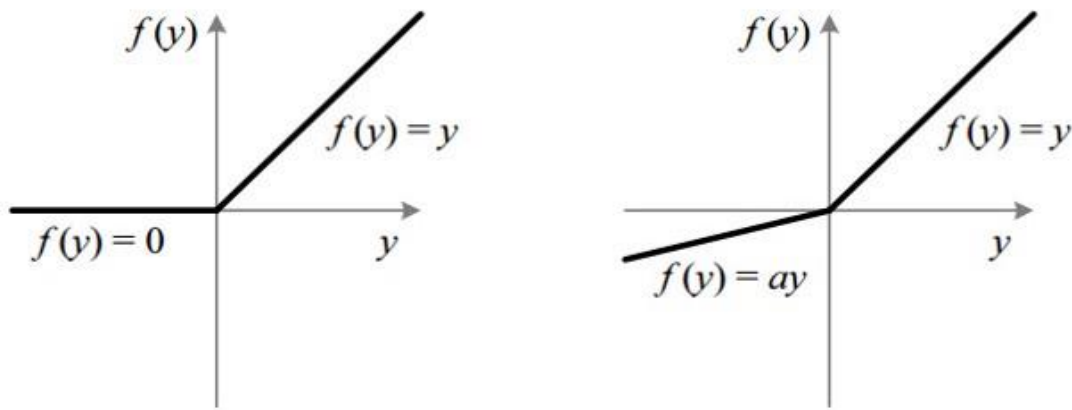
ReLU (Rectified Linear Unit) Activation Function

- **ReLU (Rectified Linear Unit) Activation Function**
 - The ReLU is the most used activation function in the world right now. Since, it is used in almost all the convolutional neural networks or deep learning.
 - ReLU is half rectified (from bottom). $f(z)$ is zero when z is less than zero and $f(z)$ is equal to z when z is above or equal to zero.
- **Range:** [0 to infinity)
- The function and its derivative **both are monotonic.**
- But the issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly.
- That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.



Leaky ReLU

- It is an attempt to solve the dying ReLU problem
- The leak helps to increase the range of the ReLU function. Usually, the value of **a** is 0.01 or so.
- When **a is not 0.01** then it is called **Randomized ReLU**.
- Therefore the **range** of the Leaky ReLU is (-infinity to infinity).
- Both Leaky and Randomized ReLU functions are monotonic in nature. Also, their derivatives also monotonic in nature.



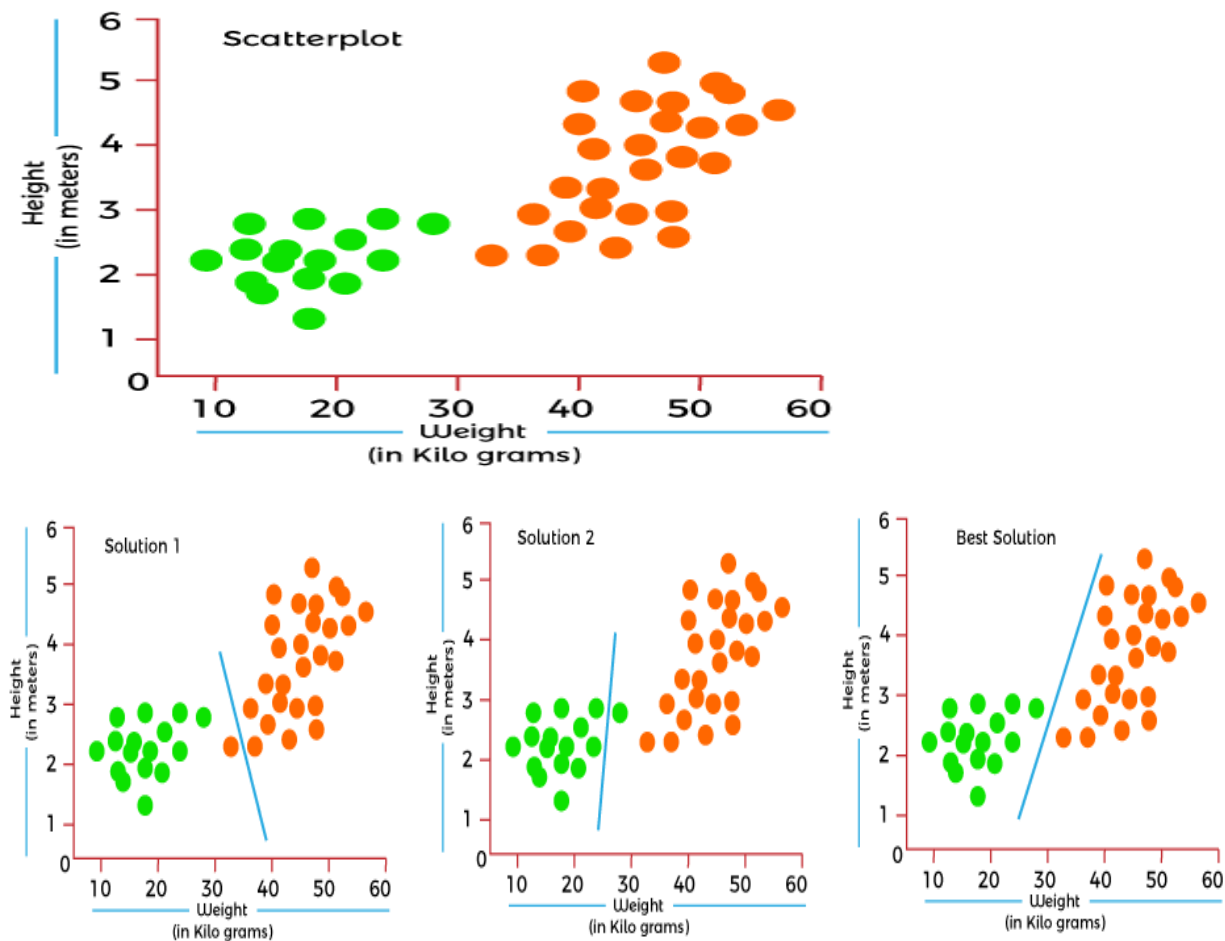
Cost function

- A Machine Learning model should have a very high level of accuracy in order to perform well with real-world applications.
- But how to calculate the accuracy of the model, i.e., how good or poor our model will perform in the real world?
- In such a case, the Cost function comes into existence.
- It is an important machine learning parameter to correctly estimate the model.
- Cost function also plays a crucial role in understanding that how well your model estimates the relationship between the input and output parameters.

COST FUNCTION IN MACHINE LEARNING



Why use Cost Function?



Gradient Descent: Minimizing the cost function

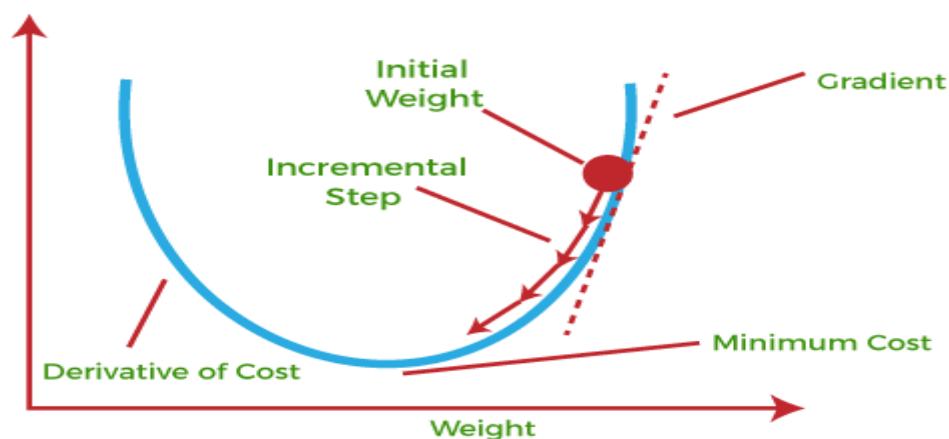
- "Gradient Descent is an optimization algorithm which is used for optimizing the cost function or error in the model."
- It enables the models to take the gradient or direction to reduce the errors by reaching to least possible error.
- Here direction refers to how model parameters should be corrected to further reduce the cost function.
- The error in your model can be different at different points, and you have to find the quickest way to minimize it, to prevent resource wastage.
- Gradient descent is an iterative process where the model gradually converges towards a minimum value, and if the model iterates further than this point, it produces little or zero changes in the loss.
- This point is known as convergence, and at this point, the error is least, and the cost function is optimized.
- *It helps in finding the local minimum of a function.*

- The best way to define the local minimum or local maximum of a function using gradient descent is as follows:
 - If we move towards a negative gradient or away from the gradient of the function at the current point, it will give the **local minimum** of that function.
 - Whenever we move towards a positive gradient or towards the gradient of the function at the current point, we will get the **local maximum** of that function.

Gradient Descent: Minimizing the cost function

To achieve this goal, it performs two steps iteratively:

- Calculates the first-order derivative of the function to compute the gradient or slope of that function.
- Move away from the direction of the gradient, which means slope increased from the current point by alpha times, where Alpha is defined as Learning Rate. It is a tuning parameter in the optimization process which helps to decide the length of the steps.



Cost function

There are many cost functions in machine learning, and each has its use cases depending on whether it is a regression problem or classification problem.

1. Regression cost Function
2. Binary Classification cost Functions
3. Multi-class Classification cost Functions

1. Regression cost Function:

Regression models deal with predicting a continuous value for example salary of an employee, price of a car, loan prediction, etc. A cost function used in the regression problem is called “Regression Cost Function”. They are calculated on the distance-based error as follows:

$$\text{Error} = y - y'$$

Where,

Y – Actual Input

Y' – Predicted output

Mean Error (ME)

- In this cost function, the error for each training data is calculated and then the mean value of all these errors is derived.
- Calculating the mean of the errors is the simplest and most intuitive way possible.
- The errors can be both negative and positive. So, they can cancel each other out during summation giving zero mean error for the model.
- Thus, this is not a recommended cost function, but it does lay the foundation for other cost functions of regression models.

Mean Squared Error (MSE)

- This improves the drawback we encountered in Mean Error above. Here a square of the difference between the actual and predicted value is calculated to avoid any possibility of negative error.
- It is measured as the average of the sum of squared differences between predictions and actual observations.
- MSE cost function
- $MSE = (\text{sum of squared errors})/n$
- It is also known as L2 loss.
- In MSE, since each error is squared, it helps to penalize even small deviations in prediction when compared to MAE. But if our dataset has outliers that contribute to larger prediction errors, then squaring this error further will magnify the error many times more and also lead to higher MSE error.
- Hence, we can say that it is less robust to outliers

Mean Absolute Error (MAE)

- This cost function also addresses the shortcoming of mean error differently.
- Here an absolute difference between the actual and predicted value is calculated to avoid any possibility of negative error.
- So in this cost function, MAE is measured as the average of the sum of absolute differences between predictions and actual observations.

- MAE Cost function
 - $MAE = (\text{sum of absolute errors})/n$
 - It is also known as L1 Loss.
 - It is robust to outliers thus it will give better results even when our dataset has noise or outliers.
- Cost functions used in classification problems are different than what we use in the regression problem. A commonly used loss function for classification is the cross-entropy loss.

Introduction to TensorFlow

Lets install Necessary Libraries

▣ Tensorflow

Pip install tensorflow==version

▣ Tensorboard

Pip install tensorboard==version

▣ Keras

Pip install keras==version

Tensorflow

- ▣ **TensorFlow** is an open-source end-to-end platform for creating Machine Learning applications.
- ▣ It is a symbolic math library that uses dataflow and differentiable programming to perform various tasks focused on training and inference of deep neural networks.
- ▣ It allows developers to create machine learning applications using various tools, libraries, and community resources.

Why is TensorFlow Popular?

- ▣ TensorFlow is the best library of all because it is built to be accessible for everyone.
- ▣ Tensorflow library incorporates different API to built at scale deep learning architecture like CNN or RNN.
- ▣ TensorFlow is based on graph computation; it allows the developer to visualize the construction of the neural network with Tensorboard.

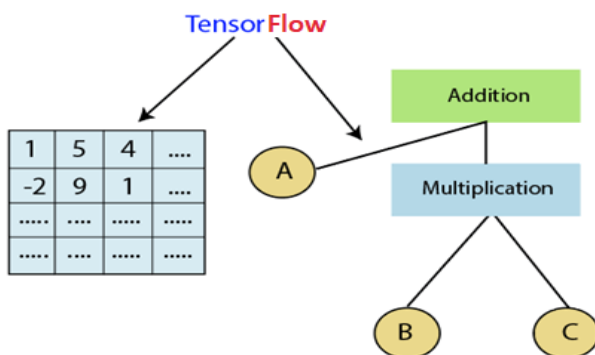
- ▣ This tool is helpful to debug the program. Finally, Tensorflow is built to be deployed at scale. It runs on CPU and GPU.
- ▣ Tensorflow attracts the largest popularity on GitHub compare to the other deep learning framework.

How TensorFlow Works

- ▣ TensorFlow enables you to build dataflow graphs and structures to define how data moves through a graph by taking inputs as a multi-dimensional array called Tensor.
- ▣ It allows you to construct a flowchart of operations that can be performed on these inputs, which goes at one end and comes at the other end as output.

The word TensorFlow is made by two words, i.e., Tensor and Flow

- ▣ **Tensor** is a multidimensional array
- ▣ **Flow** is used to define the flow of data in operation.

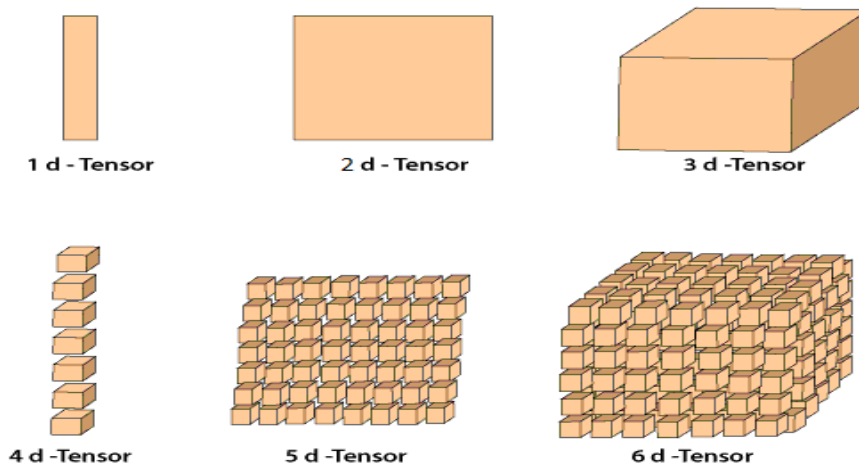


Components of TensorFlow

Tensor

- ▣ A tensor is a vector or a matrix of n-dimensional that represents all type of data.
- ▣ All values in a tensor hold similar data type with a known shape.
- ▣ The shape of the data is the dimension of the matrix or an array.
- ▣ A tensor can be generated from the input data or the result of a computation.

Dimensions of Tensor



Graph

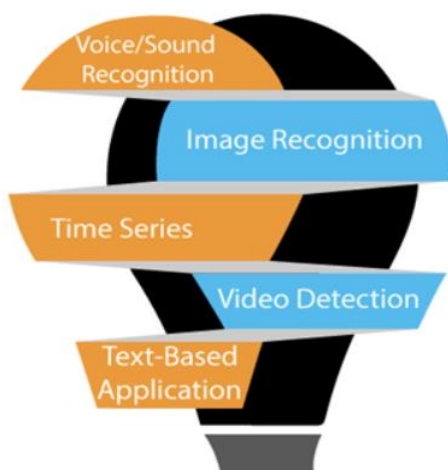
- ▣ In TensorFlow, all operations are conducted inside a graph.
- ▣ The graph is a set of calculation that takes place successively.
- ▣ The chart gathers and describes all the computations done during the training.
- ▣ Example of graph

$$X=a+b$$

$$Y=b*3$$

$$Z=X+Y$$

Applications of TensorFlow



TensorFlow Architecture

Tensorflow architecture works in three parts:

- ▣ Preprocessing the data
- ▣ Build the model
- ▣ Train and estimate the model

Lets Start working with tensorflow

- ▣ **Importing the library**

import tensorflow as tf

- ▣ **Let's create a computational graph that multiplies two numbers together.**

```
import tensorflow.compat.v1 as tf
```

```
tf.disable_v2_behavior()
```

```
X_1 = tf.placeholder(tf.float32, name = "X_1")
```

```
X_2 = tf.placeholder(tf.float32, name = "X_2")
```

```
multiply = tf.multiply(X_1, X_2, name = "multiply")
```

```
with tf.Session() as session:
```

```
    result = session.run(multiply, feed_dict={X_1:[1,2,3], X_2:[4,5,6]})
```

```
print(result)
```

Programmming Elements

- ▣ **Creation of Tensors**

```
import tensorflow as tf
```

```
x = tf.constant([[1, 2, 3, 4, 5]])
```

```
y = tf.ones((1,5))
```

```
z = tf.zeros((1,5))
```

```
q = tf.range(start=1, limit=6, delta=1)
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

```
print(q)
```

▣ Characteristics and Features of Tensor Objects

Rank

Shape

Size

Datatype

- ▣ Example: Create a Rank-3 tensor by passing a three-level nested list object to the `tf.constant` function.

```
three_level_nested_list = [[[0, 1, 2],[3, 4, 5]],  
                           [[6, 7, 8],[9, 10, 11]] ]  
  
rank_3_tensor = tf.constant(three_level_nested_list)  
print(rank_3_tensor)  
  
tensor_ndim = rank_3_tensor.ndim  
print("The number of dimensions:", tensor_ndim)
```

▣ Example for Shape

```
import tensorflow as tf  
  
three_level_nested_list = [[[0, 1, 2],[3, 4, 5]], [[6, 7, 8],[9, 10, 11]] ]  
  
rank_3_tensor = tf.constant(three_level_nested_list)  
  
tensor_shape = rank_3_tensor.shape  
print("The shape of our Tensor object is", tensor_shape)
```

Operations with Tensors

- ▣ Indexing
- ▣ Addition
- ▣ Element-wise Multiplication
- ▣ Matrix Multiplication
- ▣ Finding the Maximum or Minimum
- ▣ Finding the Index of the Max Element
- ▣ Computing Softmax Value

```
two_level_nested_list = [ [0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11] ]
```

```
rank_2_tensor = tf.constant(two_level_nested_list)
print("The 1st element of the first level is:", rank_2_tensor[0].numpy())
print("The 2nd element of the first level is:", rank_2_tensor[1].numpy())
print("The 1st element of the second level is:", rank_2_tensor[0, 0].numpy())
print("The 3rd element of the second level is:", rank_2_tensor[0, 2].numpy())
```

```
import tensorflow as tf
a = tf.constant([[2, 4],
                 [6, 8]], dtype=tf.float32)
b = tf.constant([[1, 3],
                 [5, 7]], dtype=tf.float32)
add_tensors = tf.add(a,b)
print(add_tensors)
```

```
import tensorflow as tf
a = tf.constant([.2, .5, .7, 1], dtype = tf.float64)
b = tf.constant([.1, .3, 1, 5], dtype = tf.float64)
print('a: ', a)
print('b: ', b)
res = tf.math.multiply(x = a, y = b)
print('Result: ', res)
```

```
import tensorflow as tf
a = tf.constant([[2, 4],
                 [6, 8]], dtype=tf.float32)
b = tf.constant([[1, 3],
                 [5, 7]], dtype=tf.float32)
print("The Max value of the tensor object b is:",
```

```
tf.reduce_max(b).numpy())
```

Finding the Index of the Max Element is possible with `tf.argmax()` function

```
print("The index position of the max element of the tensor object b is:",
```

```
tf.argmax(b).numpy())
```

Computing softmax is possible with `tf.nn.softmax()` function

```
print("The softmax computation result of the tensor object b is:",
```

```
tf.nn.softmax(b).numpy())
```

Manipulating Shapes

```
import tensorflow as tf
```

```
a = tf.constant([[1, 2, 3, 4, 5, 6]])
```

```
print("The shape of the initial Tensor object is:', a.shape)
```

```
b = tf.reshape(a, [6, 1])
```

```
print("The shape of the first reshaped Tensor object is:', b.shape)
```

```
c = tf.reshape(a, [3, 2])
```

```
print("The shape of the second reshaped Tensor object is:', c.shape)
```

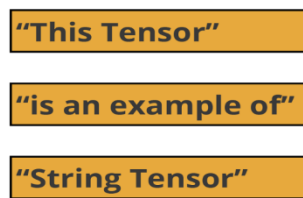
If we pass -1 passed in the `shape` argument, then we flatten our Tensor.

```
print("The shape of the flattened Tensor object is:', tf.reshape(a, [-1]))
```

Special Types of Tensors



RAGGED TENSOR



STRING TENSOR



SPARSE TENSOR

Keras

- ▣ Keras is an open-source high-level Neural Network library, which is written in Python is capable enough to run on Theano, TensorFlow, or CNTK.
- ▣ It was developed by one of the Google engineers, Francois Chollet.

- ▣ It is made user-friendly, extensible, and modular for facilitating faster experimentation with deep neural networks.
- ▣ It not only supports Convolutional Networks and Recurrent Networks individually but also their combination.

It cannot handle low-level computations, so it makes use of the **Backend** library to resolve it.

What makes Keras special?

- ▣ Focus on user experience has always been a major part of Keras.
- ▣ Large adoption in the industry.
- ▣ It is a multi backend and supports multi-platform, which helps all the encoders come together for coding.
- ▣ Research community present for Keras works amazingly with the production community.
- ▣ Easy to grasp all concepts.
- ▣ It supports fast prototyping.
- ▣ It seamlessly runs on CPU as well as GPU.
- ▣ It provides the freedom to design any architecture, which then later is utilized as an API for the project.
- ▣ It is really very simple to get started with.
- ▣ Easy production of models actually makes Keras special

Why Keras?

- ▣ **Light-weight and quick:** Few lines of keras code will achieve so much more than native Tensorflow code. You can easily design both CNN and RNNs and can run them on either GPU or CPU.
- ▣ **Emerging possible winner:** Keras is an API which runs on top of a back-end. This back-end could be either Tensorflow or Theano. Microsoft is also working to provide CNTK as a back-end to Keras.

Keras Models

- ▣ Keras has come up with two types of in-built models
- ▣ Sequential Model and an advanced Model class with functional API.
- ▣ The Sequential model tends to be one of the simplest models as it constitutes a linear set of layers.

The functional API model leads to the creation of an arbitrary network structure.

Sequential model

- ▣ The layers within the sequential models are sequentially arranged, so it is known as Sequential API.
- ▣ In most of the Artificial Neural Network, the layers are sequentially arranged, such that the data flow in between layers is in a specified sequence until it hit the output layer.
- ▣ This is how we start by importing and building a Sequential model.

```
from keras.models import Sequential
```

```
model = Sequential()
```

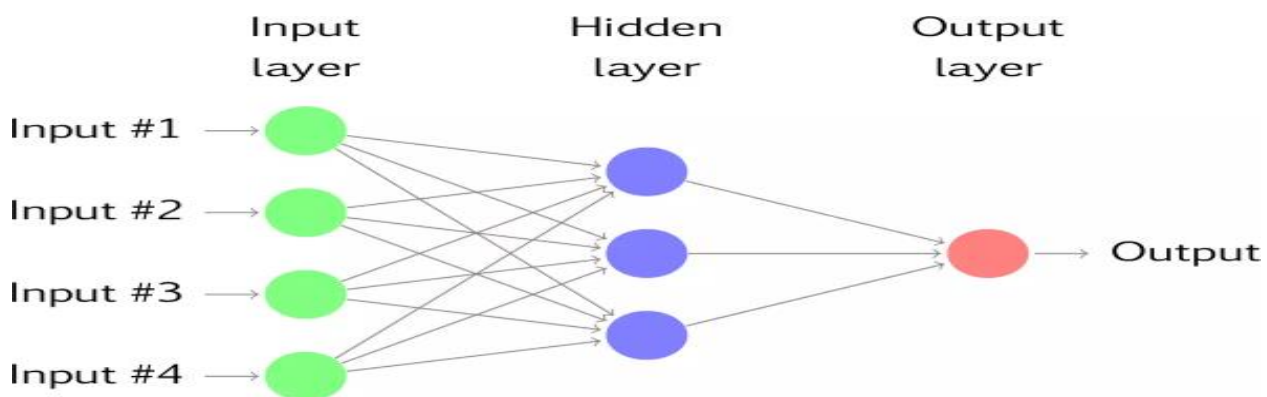
- ▣ We can add layers like Dense(fully connected layer), Activation, Conv2D, MaxPooling2D etc by calling add function.

```
from keras.layers import Dense, Activation, Conv2D, MaxPooling2D, Flatten, Dropout
```

```
model.add(Conv2D(64, (3, 3), activation='relu'))
```

Layers

- ▣ **If you stack neurons in a single line, it's called a layer; which is the next building block of neural networks.**



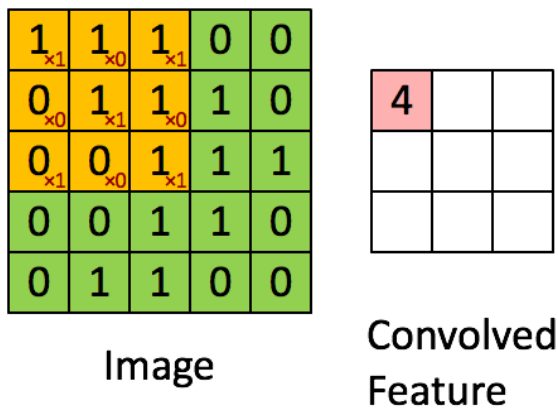
Types of layers

Typically, all the neurons in one layer, do similar kind of mathematical operations and that's how that a layer gets its name(Except for input and output layers as they do little mathematical operations)

- ▣ Convolutional Layer
- ▣ Pooling Layer
- ▣ Fully Connected Layer

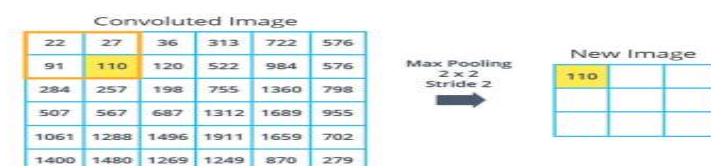
Convolutional Layer

- Convolution is a mathematical operation that's used in signal processing to filter signals, find patterns in signals etc.
- In a convolutional layer, all neurons apply convolution operation to the inputs, hence they are called convolutional neurons.



Pooling Layer

- Pooling layer is mostly used immediately after the convolutional layer to reduce the spatial size (only width and height, not depth).
- This reduces the number of parameters, hence computation is reduced. Also, less number of parameters avoid overfitting.



Fully Connected Layer

- If each neuron in a layer receives input from all the neurons in the previous layer, then this layer is called fully connected layer. The output of this layer is computed by matrix multiplication followed by bias offset.

Flatten Layer

- The last stage of a convolutional neural network (CNN) is a classifier. It is called a dense layer, which is just an artificial neural network (ANN) classifier.
- And an ANN classifier needs individual features, just like any other classifier. This means it needs a feature vector.

- ▣ Therefore, you need to convert the output of the convolutional part of the CNN into a 1D feature vector, this operation is called flattening .

Dense Layer

- ▣ A dense layer is a fully connected layer. To complete the model we can one or more dense layers.
- ▣ Once we add dense layer we call the architecture as shallow neural network.

Keras Layers

- ▣ A Keras layer requires **shape of the input (input_shape)** to understand the structure of the input data, **initializer** to set the weight for each input and finally activators to transform the output to make it non-linear.
- ▣ In between, constraints restricts and specify the range in which the weight of input data to be generated and regularizer will try to optimize the layer (and the model) by dynamically applying the penalties on the weights during optimization process.

Keras Dense Layer

- ▣ `keras.layers.Dense()`
- ▣ The dense layer can be defined as a densely-connected common Neural Network layer.
- ▣ The **output = activation(dot(input, kernel) + bias)** operation is executed by the Dense layer.
- ▣ Here an element-wise activation function is being performed by the activation, so as to pass an activation argument, a matrix of weights called **kernel** is built by the layer, and **bias** is a vector created by the layer, which is applicable only if the **use_bias** is **True**.

```
model = Sequential()
```

```
model.add(Dense(32, input_shape=(16,)))
```

```
# The model takes the input as arrays of shape (*, 16) and output arrays of shape (*, 32)
```

```
# After the first layer, you don't need to specify the size of the input:
```

```
model.add(Dense(32))
```

Shallow neural Network

```
import keras
from keras.models import Sequential
from keras.layers import Dense# Neural network
model = Sequential()
model.add(Dense(16, input_dim=20, activation='relu'))
model.add(Dense(12, activation='relu'))
model.add(Dense(4, activation='softmax'))

Model.summary()
```

CNN Model

```
import keras
from keras.models import Sequential
from keras.layers import Dense,Conv2D,MaxPooling2D,Flatten
# Neural network
cnn =Sequential([
    Conv2D(filters=32, kernel_size=(3, 3), activation='relu',
input_shape=(32, 32, 3)),MaxPooling2D((2, 2)),
    Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
cnn.summary()
```

Keras Loss Function

- ▣ While optimization, we use a function to evaluate the weights and try to minimize the error.
- ▣ This objective function is our loss function and the evaluation score calculated by this loss function is called loss.

- ▣ In simple words, losses refer to the quality that is computed by the model and try to minimize during model training.
- ▣ This loss function has a very important role as the improvement in its evaluation score means a better network.

Probabilistic Losses

- ▣ Binary Crossentropy
- ▣ Categorical Crossentropy
- ▣ Sparse Categorical Crossentropy
- ▣ Poisson
- ▣ Kullback-Leibler Divergence

Binary Cross-Entropy(BCE) loss

- ▣ BCE is used to compute the cross-entropy between the true labels and predicted outputs, it is majorly used when there are only two label classes problems arrived like dog and cat classification(0 or 1), for each example, it outputs a single floating value per prediction.

```
y_true = [[0., 1.], [0., 0.]]
```

```
y_pred = [[0.5, 0.4], [0.4, 0.5]]
```

```
bce_loss = tf.keras.losses.BinaryCrossentropy() bce_loss(y_true, y_pred).numpy()
```

Categorical Crossentropy loss

- ▣ The categorical cross-entropy loss function is used to compute loss between labels and prediction, it is used when there are two or more label classes present in our problem use case like animal classification: cat, dog, elephant, horse, etc. Also if you ever want to use labels as integers, you can this loss functions confidently.

```
y_true = [[0, 1, 0], [0, 0, 1]]
```

```
y_pred = [[0.05, 0.95, 0], [0.1, 0.8, 0.1]]
```

```
cce_loss = tf.keras.losses.CategoricalCrossentropy() cce_loss(y_true, y_pred).numpy()
```

Regression losse

- ▣ Mean Squared error
- ▣ Mean Absolute Error
- ▣ Mean Absolute percentage error

- ▣ Mean Squared Logarithmic Error
- ▣ Cosine Similarity
- ▣ Huber
- ▣ Log Cosh

Mean Squared Error(MSE)

- ▣ MSE tells you how close a regression line is to a set of points. And It does this by taking the distances from the points to the regression line and squaring them. The squaring is a must as it removes the negative signs from the problem. MSE also gives more weight to larger differences which are called the mean squared error.

```
y_true = [[0., 1.], [0., 0.]]
```

```
y_pred = [[1., 1.], [1., 0.]]
```

```
mse = tf.keras.losses.MeanSquaredError()
```

```
mse(y_true, y_pred).numpy()
```

MeanAbsoluteError

- ▣ It is the difference between the measured value and the “true” value. For example, if a scale states 80 kg but you know your true weight is 79 kg , then the scale has an absolute error of $80 \text{ kg} - 79 \text{ kg} = 1 \text{ kg}$

```
y_true = [[0., 1.], [0., 0.]]
```

```
y_pred = [[1., 1.], [1., 0.]]
```

```
mae = tf.keras.losses.MeanAbsoluteError() mae(y_true, y_pred).numpy()
```

Keras Optimizers

- ▣ Optimizers are the expanded class, which includes the method to train your machine/deep learning model.
- ▣ Right optimizers are necessary for your model as they improve training speed and performance
- ▣ During the training of the model, we tune the parameters(also known as hyperparameter tuning) and weights to minimize the loss and try to make our prediction accuracy as correct as possible
- ▣ Now to change these parameters the optimizer’s role came in, which ties the model parameters with the loss function by updating the model in response to the loss function output.

- ▣ Simply optimizers shape the model into its most accurate form by playing with model weights.

AdaGrad Optimizer

- ▣ Adagrad adapts the learning rate specifically with individual features
- ▣ It means that some of the weights in your dataset have different learning rates than others.
- ▣ It always works best in a sparse dataset where a lot of inputs are missing.
- ▣ In TensorFlow, you can call the optimizer using the below command

```
tf.keras.optimizers.Adagrad(learning_rate=0.001, initial_accumulator_value=0.1, epsilon=1e-07, name="Adagrad", **kwargs )
```

RMSprop Optimizer

- ▣ It is an exclusive version of Adagrad developed by Geoffrey Hinton, now the thinking behind this optimizer was pretty straight forward.
- ▣ Instead of letting all of the gradients accumulate for momentum, it only accumulates gradients in a specific fix window.
- ▣ It is exactly like Adaprop(an updated version of Adagrad with some improvement), you can call this in the TensorFlow framework using the below command.

```
tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9, momentum=0.0, epsilon=1e-07, centered=False, name='RMSprop', **kwargs )
```

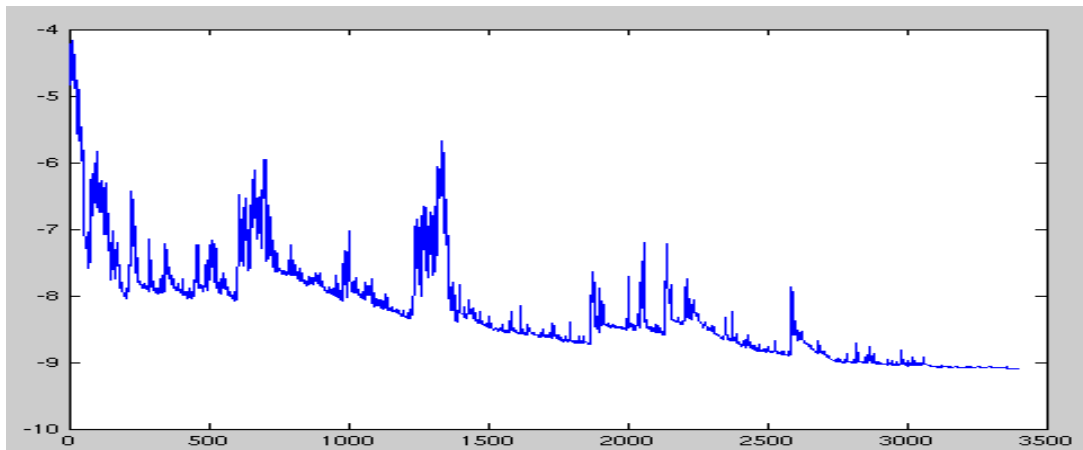
Adam Optimizer

- ▣ Adam stands for adaptive moment estimation, which is another way of using past gradients to calculate current gradients, Adam utilizes the concept of momentum by adding fractions of previous gradients to the current one, it is practically accepted in many projects during training neural nets.

```
tf.keras.optimizers.Adam( learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07, amsgrad=False, name='Adam', **kwargs )
```

SGD Optimizer

- ▣ SGD performs redundant computations for bigger datasets, as it recomputes gradients for the same example before each parameter update. It performs frequent updates with a high variance that cause the objective function to fluctuate heavily as shown in below image.



```
tf.keras.optimizers.SGD(learning_rate=0.01,  
momentum=0.0, nesterov=False, name="SGD", **kwargs )
```

Keras Metrics

- ▣ A metric is **a function that is used to judge the performance of your model.**
- ▣ Metric functions are similar to loss functions, except that the results from evaluating a metric are not used when training the model.
- ▣ Note that you may use any loss function as a metric

Keras Regression Metrics

- ▣ Mean Squared Error: `mean_squared_error`, MSE or mse
- ▣ Mean Absolute Error: `mean_absolute_error`, MAE, mae
- ▣ Mean Absolute Percentage Error: `mean_absolute_percentage_error`, MAPE, mape
- ▣ Cosine Proximity: `cosine_proximity`, cosine

Keras Classification Metrics

- ▣ **Binary Accuracy:** `binary_accuracy`, acc
- ▣ **Categorical Accuracy:** `categorical_accuracy`, acc
- ▣ **Sparse Categorical Accuracy:** `sparse_categorical_accuracy`
- ▣ **Top k Categorical Accuracy:** `top_k_categorical_accuracy` (requires you specify a k parameter)

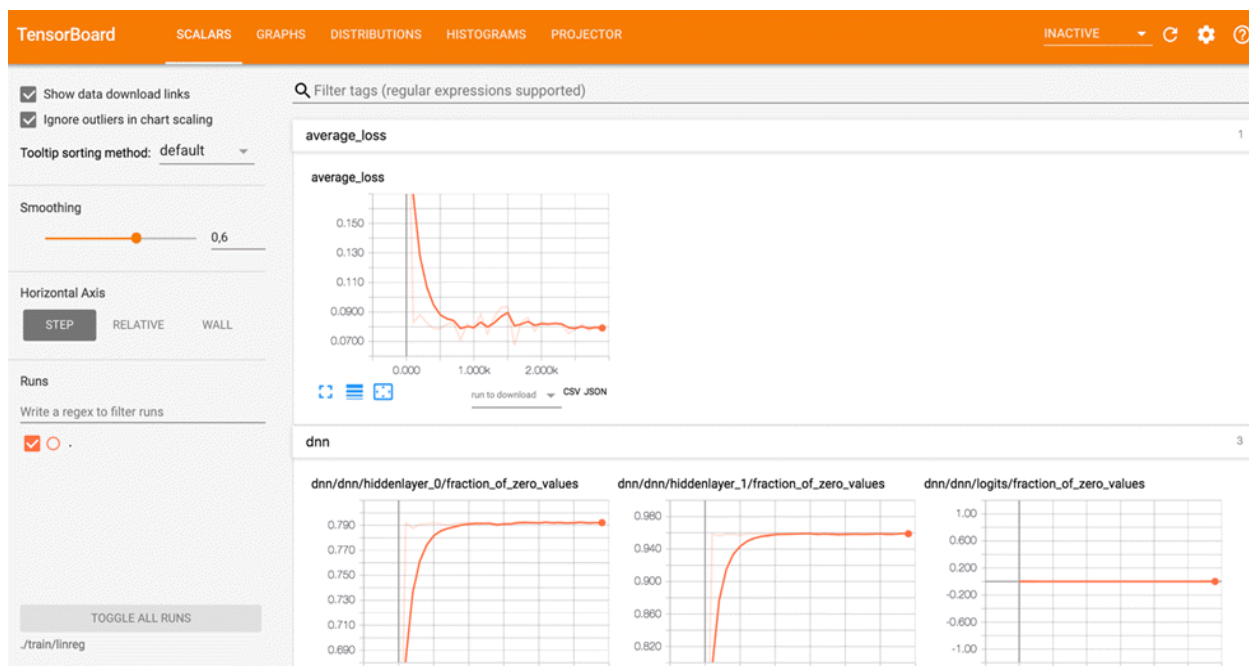
- ▣ **Sparse Top k Categorical Accuracy:** `sparse_top_k_categorical_accuracy` (requires you specify a k parameter)

Keras Callbacks

- ▣ Building Deep Learning models without callbacks is like driving a car with no functioning brakes.
- ▣ You have little to no control over the whole process that is very likely to result in a disaster.
- ▣ A callback is a set of functions to be applied at given stages of the training procedure. You can use callbacks to get a view on internal states and statistics of the model during training.

Tensorboard

- ▣ **TensorBoard** is the interface used to visualize the graph and other tools to understand, debug, and optimize the model.
- ▣ It is a tool that provides measurements and visualizations for machine learning workflow.
- ▣ It helps to track metrics like loss and accuracy, model graph visualization, project embedding at lower-dimensional spaces, etc.



- ▣ **Scalars:** Show different useful information during the model training
- ▣ **Graphs:** Show the model
- ▣ **Histogram:** Display weights with a histogram
- ▣ **Distribution:** Display the distribution of the weight

- ▣ **Projector:** Show Principal component analysis and T-SNE algorithm. The technique used for dimensionality reduction
- ▣ The basic idea behind tensorboard is that neural network can be something known as a black box and we need a tool to inspect what's inside this box.
- ▣ You can imagine tensorboard as a flashlight to start dive into the neural network.
- ▣ It helps to understand the dependencies between operations, how the weights are computed, displays the loss function and much other useful information.
- ▣ When you bring all these pieces of information together, you have a great tool to debug and find how to improve the model.

Set up tensorboard

```
import tensorflow as tf

mnist = tf.keras.datasets.mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0

model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')])

model.compile(optimizer='sgd',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

log_folder = 'C:\Users\Prakash\Desktop\logs'

from tensorflow.keras.callbacks import TensorBoard

callbacks = [TensorBoard(log_dir=log_folder,
                        histogram_freq=1,
                        write_graph=True,
                        write_images=True,
                        update_freq='epoch',
```

```

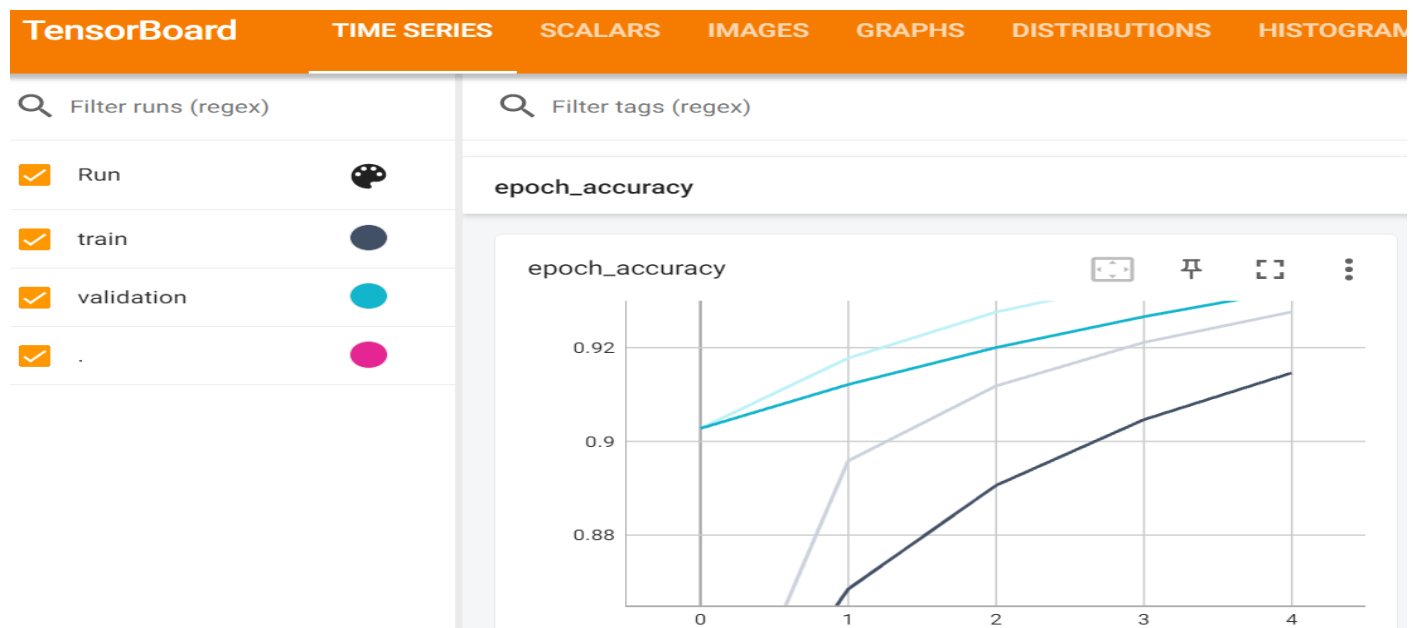
        profile_batch=2,
        embeddings_freq=1)]

model.fit(X_train, y_train,
        epochs=10,
        validation_split=0.2,
        callbacks=callbacks)

model.save("model.h5")

```

Tensorboard Accuracy graph



Tensorboard Loss graph

